

Lab 11.

Local Features, SIFT, Matching, and Image Alignment with RANSAC

Computer Vision, MIM, HUS

2025-2026

Contents

1	Introduction	3
1.1	Learning objectives	3
1.2	Expected background	4
1.3	Suggested image collection	4
2	From local image structure to local features	4
2.1	Why local features matter	4
2.2	Detector versus descriptor	4
2.3	Desired invariance properties	5
3	Harris corner detection	5
3.1	Intuition	5
3.2	From image gradients to the structure tensor	5
3.3	Harris response	5
3.4	Practical pipeline	6
3.5	What information does a Harris feature contain?	6
3.6	NumPy implementation cell	6
4	Scale-space features: LoG and DoG	8
4.1	Why scale matters	8
4.2	Laplacian of Gaussian (LoG)	9
4.3	Difference of Gaussian (DoG)	9
4.4	Scale-space extrema	9
4.5	What information does a LoG or DoG keypoint contain?	9
4.6	NumPy implementation cell	10
5	From detection to description	11
5.1	Why detection alone is insufficient	11
5.2	Simple patch descriptors	11
5.3	Gradient-based descriptors	12
6	SIFT features	12
6.1	Overview	12
6.2	SIFT detection stages	12
6.3	Orientation assignment	12
6.4	Descriptor construction	13
6.5	What information does a SIFT feature contain?	13
6.6	Simplified SIFT-like descriptor in NumPy	13
6.7	OpenCV SIFT implementation	14

7	Feature matching	15
7.1	Distance between descriptors	15
7.2	Nearest-neighbor and kNN matching	15
7.3	Lowe's ratio test	15
7.4	What information does a match contain?	16
8	RANSAC for robust geometric estimation	16
8.1	Why RANSAC is needed	16
8.2	Key ideas	17
8.3	Homography model	17
8.4	RANSAC pipeline	17
8.5	Reprojection error	17
8.6	Synthetic NumPy demonstration	17
8.7	OpenCV homography with RANSAC	19
9	Application: image alignment and basic panorama	20
9.1	Full pipeline overview	20
9.2	OpenCV panoramic alignment cell	20
9.3	Failure cases to discuss	21
10	Suggested structure of the accompanying notebook	21
11	Evaluation and reporting guidelines	22
12	Comprehensive final exercises	22
13	Submission checklist	22
14	Concluding remarks	23

1. Introduction

This lab manual develops a complete classical computer vision pipeline beginning with local image structure and ending with geometric image alignment. The lab is deliberately positioned after students have already studied grayscale and color image processing, filtering, gradients, edge detection, basic segmentation, geometric transforms, and introductory notions of image features. The main goal is to transform that knowledge into a feature-centric view of vision: instead of treating an image only as a dense array of pixel values, students learn how to identify informative locations, describe the visual content around these locations, match them across images, and robustly estimate geometric transformations.

The lab focuses on the following building blocks:

- Harris corner detection as a classic interest point detector,
- Laplacian of Gaussian (LoG) and Difference of Gaussian (DoG) as scale-space feature detectors,
- local descriptors, especially SIFT,
- descriptor matching and geometric verification,
- RANSAC for robust model estimation,
- image alignment and basic panoramic stitching.

The pedagogical structure is intentionally dual:

1. **Transparent NumPy implementations** to reveal the algorithmic pipeline step by step;
2. **OpenCV implementations** to connect the same ideas to practical, full-scale computer vision applications.

1.1. Learning objectives

After completing this lab, students should be able to:

- distinguish between a detector and a descriptor;
- explain why corners, blobs, and scale-space extrema are useful for matching;
- implement Harris response computation from image gradients and the structure tensor;
- construct a simple Gaussian pyramid and DoG pyramid;
- interpret the information stored in local features such as position, scale, response, orientation, and descriptor vector;
- explain the internal structure of SIFT descriptors and the role of histogramming gradient orientations;
- perform feature matching and apply Lowe's ratio test;
- explain RANSAC conceptually and implement or use it for robust homography estimation;
- build a complete image alignment pipeline and analyze failure cases.

1.2. Expected background

Students are assumed to know:

- image representation and basic image I/O,
- grayscale conversion and filtering,
- Gaussian smoothing, Sobel filters, and gradients,
- convolution and local neighborhoods,
- introductory feature notions (global versus local),
- the high-level ideas of Harris, LoG, and DoG from lectures.

1.3. Suggested image collection

For good experiments, students should prepare image pairs from the following categories:

- building facades with many corners and repeated windows,
- books, posters, magazine covers, or product boxes,
- indoor scenes with rich texture and overlap between two views,
- scenes with circular or blob-like objects such as coins, buttons, lamps, or plates,
- two partially overlapping photographs for panorama stitching.

Avoid images that are too smooth, overly blurred, or contain very little overlap.

2. From local image structure to local features

2.1. Why local features matter

Global image representations often summarize the entire image into one vector. Such representations are useful for coarse retrieval or scene classification, but they are not naturally suited for pixel-level geometric correspondence. If one wants to match two views of the same object or estimate how one image should be warped onto another, one must work with local structures that can be re-identified reliably.

A local feature is usually associated with a spatial location and, in stronger formulations, with a characteristic scale and orientation. A local feature becomes particularly useful when it is both *repeatable* and *distinctive*. Repeatability means the feature can be found again under imaging changes. Distinctiveness means the feature's local appearance is rich enough to avoid confusion with other locations.

2.2. Detector versus descriptor

A detector answers the question:

Where in the image should we pay attention?

A descriptor answers the question:

How should we numerically describe the local appearance around a detected point?

This distinction is fundamental:

- Harris is mainly a detector.
- LoG and DoG are mainly detectors in scale-space.
- SIFT includes both a detector and a descriptor.

2.3. Desired invariance properties

A strong local feature pipeline should be robust to:

- small image noise,
- slight changes in illumination,
- translation,
- rotation,
- moderate scale changes,
- some local perspective variation.

No method is perfect under all transformations, so part of the lab is to observe which methods survive which changes.

3. Harris corner detection

3.1. Intuition

A corner is a location where image intensity changes significantly in more than one direction. This makes corners more informative for matching than flat regions and often more stable than pure edges. On an edge, shifting a small window along the edge may not change the appearance much. At a corner, shifting the window in almost any direction changes the local appearance noticeably.

3.2. From image gradients to the structure tensor

Let $I(x, y)$ denote a grayscale image. Compute image gradients:

$$I_x = \frac{\partial I}{\partial x}, \quad I_y = \frac{\partial I}{\partial y}.$$

For a local window around a pixel, Harris measures how much the image changes when the window is shifted slightly. A second-order approximation yields the *structure tensor*:

$$M(x, y) = \begin{bmatrix} S_{xx}(x, y) & S_{xy}(x, y) \\ S_{xy}(x, y) & S_{yy}(x, y) \end{bmatrix}$$

with

$$S_{xx} = G_\sigma * I_x^2, \quad S_{yy} = G_\sigma * I_y^2, \quad S_{xy} = G_\sigma * (I_x I_y),$$

where G_σ is a Gaussian smoothing kernel.

The eigenvalues of M summarize directional gradient strength:

- both small: flat region,
- one large and one small: edge,
- both large: corner.

3.3. Harris response

Instead of explicitly computing eigenvalues at every pixel, Harris proposed the response:

$$R = \det(M) - k(\text{trace}(M))^2,$$

where

$$\det(M) = S_{xx}S_{yy} - S_{xy}^2, \quad \text{trace}(M) = S_{xx} + S_{yy}.$$

The empirical constant k is usually chosen around 0.04 to 0.06.

3.4. Practical pipeline

Algorithm 1 Harris corner detection

- 1: Convert image to grayscale and floating point
 - 2: Compute I_x and I_y
 - 3: Form I_x^2 , I_y^2 , and $I_x I_y$
 - 4: Smooth these products with a Gaussian filter
 - 5: Compute $R = \det(M) - k(\text{trace}(M))^2$
 - 6: Threshold the response map
 - 7: Apply non-maximum suppression
 - 8: Return corner coordinates and responses
-

3.5. What information does a Harris feature contain?

A Harris point is not yet a full descriptor. It typically stores:

- image location (x, y) ,
- response value R ,
- sometimes a local patch or support window if one later constructs a descriptor around it.

This distinction is important: Harris tells us *where* an interesting point is, but not a highly invariant descriptor by itself.

3.6. NumPy implementation cell

Listing 1: Notebook Cell: Harris corner detection with NumPy

```

1 # Cell 1 - Harris corner detection with NumPy
2 import cv2
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 def show_image(title, img, cmap=None, figsize=(6, 5)):
7     plt.figure(figsize=figsize)
8     if img.ndim == 2:
9         plt.imshow(img, cmap=cmap or "gray")
10    else:
11        plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
12    plt.title(title)
13    plt.axis("off")
14    plt.show()
15
16 def conv2d_same(image, kernel):
17     kh, kw = kernel.shape
18     ph, pw = kh // 2, kw // 2
19     padded = np.pad(image, ((ph, ph), (pw, pw)), mode="reflect")
20     out = np.zeros_like(image, dtype=np.float32)
21     kernel_flipped = np.flipud(np.fliplr(kernel)).astype(np.float32)
22     for y in range(image.shape[0]):
23         for x in range(image.shape[1]):
24             patch = padded[y:y+kh, x:x+kw]
25             out[y, x] = np.sum(patch * kernel_flipped)
26     return out
27
28 def gaussian_kernel(size=5, sigma=1.0):
29     ax = np.arange(-(size//2), size//2 + 1)

```

```

30     xx, yy = np.meshgrid(ax, ax)
31     kernel = np.exp(-(xx**2 + yy**2) / (2 * sigma**2))
32     kernel /= np.sum(kernel)
33     return kernel.astype(np.float32)
34
35 def nonmax_suppression(response, threshold_ratio=0.01, window_size=5):
36     R = response.copy()
37     thresh = threshold_ratio * R.max()
38     coords = []
39     half = window_size // 2
40     for y in range(half, R.shape[0] - half):
41         for x in range(half, R.shape[1] - half):
42             val = R[y, x]
43             if val > thresh:
44                 patch = R[y-half:y+half+1, x-half:x+half+1]
45                 if val == np.max(patch):
46                     coords.append((x, y, float(val)))
47     return coords
48
49 img_bgr = cv2.imread("data/harris_building.jpg")
50 img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY).astype(np.float32) /
51     255.0
52
53 sobel_x = np.array([[ -1, 0, 1],
54                     [ -2, 0, 2],
55                     [ -1, 0, 1]], dtype=np.float32)
56
57 sobel_y = sobel_x.T
58
59 Ix = conv2d_same(img_gray, sobel_x)
60 Iy = conv2d_same(img_gray, sobel_y)
61
62 Ixx = Ix * Ix
63 Iyy = Iy * Iy
64 Ixy = Ix * Iy
65
66 G = gaussian_kernel(size=5, sigma=1.2)
67 Sxx = conv2d_same(Ixx, G)
68 Syy = conv2d_same(Iyy, G)
69 Sxy = conv2d_same(Ixy, G)
70
71 k = 0.04
72 R = (Sxx * Syy - Sxy**2) - k * (Sxx + Syy)**2
73 corners = nonmax_suppression(R, threshold_ratio=0.02, window_size=5)
74
75 vis = img_bgr.copy()
76 for x, y, r in corners:
77     cv2.circle(vis, (x, y), 2, (0, 0, 255), -1)
78
79 print("Number of detected Harris corners:", len(corners))
80 print("First 10 corners [x, y, response]:")
81 for item in corners[:10]:
82     print(item)
83
84 show_image("Harris response (normalized)", (R - R.min()) / (R.max() - R.
85     min()), cmap="inferno")
86 show_image("Detected Harris corners", vis)

```

Expected results. On structured scenes such as building facades, windows, bookshelves, or checkerboards, the strongest responses should appear near intersections and corner-like intensity changes. Flat walls should produce near-zero response. Long straight edges may produce weaker or unstable responses.

Discussion. Students should compare the Harris response map with the image gradient maps. A valuable observation is that Harris does not operate directly on intensity values alone: it relies on the local second-moment behavior induced by image gradients. This connects the method tightly to earlier topics such as convolution, smoothing, and gradient estimation.

Listing 2: Notebook Cell: Harris via OpenCV

```

1 # Cell 2 - Harris via OpenCV
2 img_bgr = cv2.imread("data/harris_building.jpg")
3 gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
4 gray_f = np.float32(gray)
5
6 R_cv = cv2.cornerHarris(gray_f, blockSize=2, ksize=3, k=0.04)
7 R_cv = cv2.dilate(R_cv, None)
8
9 vis_cv = img_bgr.copy()
10 vis_cv[R_cv > 0.01 * R_cv.max()] = [0, 0, 255]
11
12 print("OpenCV Harris response shape:", R_cv.shape)
13 show_image("OpenCV Harris corners", vis_cv)
14
15 pts = cv2.goodFeaturesToTrack(gray, maxCorners=200, qualityLevel=0.01,
16                               minDistance=8)
17 vis_gftt = img_bgr.copy()
18 if pts is not None:
19     pts = np.int32(pts)
20     print("Number of goodFeaturesToTrack points:", len(pts))
21     for p in pts:
22         x, y = p.ravel()
23         cv2.circle(vis_gftt, (x, y), 3, (0, 255, 0), -1)
24
25 show_image("OpenCV goodFeaturesToTrack", vis_gftt)

```

Exercises after Harris.

1. Change the smoothing scale in the structure tensor and study how the number and distribution of corners change.
2. Replace the Sobel operator with Scharr or a manually designed derivative filter and compare the response maps.
3. Rotate the image by 30° , 60° , and 90° . Are the strongest corners preserved?
4. Add Gaussian noise or blur. Which type of degradation hurts Harris more?

4. Scale-space features: LoG and DoG

4.1. Why scale matters

A detector that operates at a single fixed neighborhood size cannot easily handle the case where the same structure appears at different magnifications. A small corner may disappear if the smoothing

window is too large; a large blob may not be properly captured if the support is too small. This motivates *scale-space*, where the image is analyzed at multiple scales.

4.2. Laplacian of Gaussian (LoG)

The Laplacian operator is the sum of second derivatives:

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}.$$

Applying the Laplacian after Gaussian smoothing yields the Laplacian of Gaussian:

$$LoG(x, y, \sigma) = \nabla^2(G_\sigma * I).$$

In blob detection, one often uses the scale-normalized form

$$\sigma^2 \nabla^2(G_\sigma * I),$$

because it makes responses more comparable across scales.

4.3. Difference of Gaussian (DoG)

DoG is defined as the difference of two Gaussian-smoothed images at nearby scales:

$$DoG(x, y, \sigma) = G_{k\sigma} * I - G_\sigma * I.$$

DoG is widely used because it approximates the scale-normalized LoG efficiently and forms the basis of the original SIFT detector.

4.4. Scale-space extrema

A feature in scale-space is not just a spatial point. It is a point in a three-dimensional domain:

$$(x, y, \sigma).$$

A candidate keypoint is identified when its response is a local maximum or minimum relative to neighboring pixels in the same scale and neighboring scales.

4.5. What information does a LoG or DoG keypoint contain?

A detected scale-space keypoint usually includes:

- position (x, y) ,
- scale parameter σ ,
- response magnitude,
- possibly the sign of the extremum (bright blob versus dark blob),
- a corresponding support region whose radius is related to scale.

Unlike Harris, such features explicitly encode *how large* the local structure is.

4.6. NumPy implementation cell

Listing 3: Notebook Cell: DoG scale-space with NumPy

```

1 # Cell 3 - DoG scale-space with NumPy
2 import cv2
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 def build_gaussian_pyramid(gray, sigmas):
7     pyr = []
8     for s in sigmas:
9         blurred = cv2.GaussianBlur(gray, (0, 0), sigmaX=s, sigmaY=s)
10        pyr.append(blurred.astype(np.float32))
11    return pyr
12
13 def build_dog_pyramid(gauss_pyr):
14     dogs = []
15     for i in range(len(gauss_pyr) - 1):
16         dogs.append(gauss_pyr[i+1] - gauss_pyr[i])
17    return dogs
18
19 def detect_dog_extrema(dogs, threshold=0.03):
20     keypoints = []
21     for s in range(1, len(dogs)-1):
22         prev_img, cur_img, next_img = dogs[s-1], dogs[s], dogs[s+1]
23         H, W = cur_img.shape
24         for y in range(1, H-1):
25             for x in range(1, W-1):
26                 val = cur_img[y, x]
27                 if abs(val) < threshold:
28                     continue
29                 cube = np.stack([
30                     prev_img[y-1:y+2, x-1:x+2],
31                     cur_img[y-1:y+2, x-1:x+2],
32                     next_img[y-1:y+2, x-1:x+2]
33                 ], axis=0)
34                 if val == cube.max() or val == cube.min():
35                     keypoints.append((x, y, s, float(val)))
36    return keypoints
37
38 img = cv2.imread("data/blob_scene.jpg", cv2.IMREAD_GRAYSCALE).astype(np.
39 float32) / 255.0
40 sigmas = [1.0, 1.6, 2.5, 4.0, 6.4]
41 gauss_pyr = build_gaussian_pyramid(img, sigmas)
42 dogs = build_dog_pyramid(gauss_pyr)
43 kps = detect_dog_extrema(dogs, threshold=0.02)
44
45 vis = cv2.cvtColor((img * 255).astype(np.uint8), cv2.COLOR_GRAY2BGR)
46 for x, y, s_idx, resp in kps[:300]:
47     radius = max(2, int(sigmas[s_idx] * 2))
48     color = (0, 255, 0) if resp > 0 else (0, 0, 255)
49     cv2.circle(vis, (x, y), radius, color, 1)
50
51 print("Number of DoG extrema:", len(kps))
52 print("First 10 keypoints as (x, y, scale_index, response):")
53 for item in kps[:10]:
54     print(item)
55
56 show_image("DoG-based scale-space detections", vis)

```

Expected results. Images containing circular objects or repeated structures at different sizes should produce detections with visibly different support radii. Students should see that scale-space detections are more appropriate than purely corner-based methods when object size varies.

Discussion. A key conceptual shift occurs here: local features are no longer points only in the image plane. They are points in a spatial-scale volume. This is one of the major reasons SIFT achieves scale robustness.

Exercises after LoG/DoG.

1. Vary the scale progression factor k and study the effect on detected extrema.
2. Plot the number of extrema detected at each scale level.
3. Compare detections on the original image and a downsampled or upsampled version of the same scene.
4. Implement a simple LoG response and compare its detections to the DoG approximation.

5. From detection to description

5.1. Why detection alone is insufficient

Suppose Harris or DoG gives a set of candidate points in two images. If all we know is the location of each point, we still do not know which point in image A corresponds to which point in image B. We need a numerical summary of the local appearance around each keypoint.

A descriptor should satisfy two competing requirements:

- it must be distinctive enough to separate one local region from many others;
- it must remain reasonably stable under changes in viewpoint, scale, rotation, and illumination.

5.2. Simple patch descriptors

A basic descriptor can be built by extracting a small patch around the keypoint, flattening it into a vector, and optionally normalizing it. This is pedagogically useful but limited. Such patch descriptors are sensitive to rotation, scale, and misalignment. Their main value in the lab is as a baseline to motivate more sophisticated descriptors.

Listing 4: Notebook Cell: simple patch descriptor for Harris points

```

1 # Cell 4 - Simple patch descriptor around Harris points
2 def extract_patch_descriptor(gray, x, y, patch_size=11):
3     half = patch_size // 2
4     if y - half < 0 or x - half < 0 or y + half >= gray.shape[0] or x +
        half >= gray.shape[1]:
5         return None
6     patch = gray[y-half:y+half+1, x-half:x+half+1].astype(np.float32)
7     patch = patch - patch.mean()
8     norm = np.linalg.norm(patch) + 1e-8
9     patch = patch / norm
10    return patch.flatten()
11
12 gray = cv2.imread("data/harris_building.jpg", cv2.IMREAD_GRAYSCALE).
    astype(np.float32) / 255.0

```

```

13 example_descriptors = []
14 for x, y, r in corners[:20]:
15     desc = extract_patch_descriptor(gray, x, y, patch_size=11)
16     if desc is not None:
17         example_descriptors.append((x, y, desc))
18
19 print("Number of valid patch descriptors:", len(example_descriptors))
20 if example_descriptors:
21     x0, y0, d0 = example_descriptors[0]
22     print("Example descriptor location:", (x0, y0))
23     print("Descriptor shape:", d0.shape)
24     print("First 20 descriptor values:", d0[:20])

```

5.3. Gradient-based descriptors

Gradient-based descriptors encode local edge orientation statistics rather than raw intensity values. This is advantageous because:

- gradients are more stable under additive brightness shifts;
- orientation histograms summarize local structure compactly;
- a canonical orientation can make the descriptor more rotation invariant.

This idea culminates in SIFT.

6. SIFT features

6.1. Overview

SIFT, or Scale-Invariant Feature Transform, is one of the most influential classical local feature methods. It combines:

- a detector based on DoG extrema in scale-space,
- a descriptor based on local histograms of gradient orientation.

6.2. SIFT detection stages

The original SIFT pipeline involves:

1. build Gaussian and DoG pyramids;
2. find 3D extrema across space and scale;
3. refine keypoint localization;
4. reject unstable low-contrast points and edge-like responses;
5. assign one or more canonical orientations.

6.3. Orientation assignment

A keypoint must be assigned a dominant orientation to achieve rotation robustness. Around each keypoint, gradients are computed, weighted by a Gaussian window, and accumulated into an orientation histogram. The highest peak defines the main orientation. Additional orientations may be used for secondary keypoints if peaks are sufficiently strong.

6.4. Descriptor construction

For each oriented keypoint:

- take a local support region around the keypoint;
- rotate it into the canonical orientation frame;
- divide it into a 4×4 grid of cells;
- compute an 8-bin orientation histogram in each cell;
- concatenate all cell histograms to form a vector of length

$$4 \times 4 \times 8 = 128.$$

The descriptor is then normalized, and often thresholded and renormalized, which improves robustness to illumination changes.

6.5. What information does a SIFT feature contain?

For each detected SIFT keypoint, one can usually inspect:

- **pt**: the image position (x, y) ,
- **size**: the characteristic scale,
- **angle**: the assigned dominant orientation,
- **response**: saliency measure,
- **octave**: pyramid level information,
- **descriptor**: a 128-dimensional vector.

This is the ideal section where students should be asked to print keypoint metadata and inspect descriptor vectors directly.

6.6. Simplified SIFT-like descriptor in NumPy

Listing 5: Notebook Cell: simplified SIFT-like descriptor with NumPy

```

1 # Cell 5 - Simplified SIFT-like descriptor
2 def gradient_maps(gray):
3     gx = cv2.Sobel(gray, cv2.CV_32F, 1, 0, ksize=3)
4     gy = cv2.Sobel(gray, cv2.CV_32F, 0, 1, ksize=3)
5     mag = np.sqrt(gx * gx + gy * gy)
6     ang = (np.rad2deg(np.arctan2(gy, gx)) + 360.0) % 360.0
7     return mag, ang
8
9 def sift_like_descriptor(gray, x, y, patch_size=16, cell_size=4, num_bins
    =8):
10     half = patch_size // 2
11     if y - half < 0 or x - half < 0 or y + half >= gray.shape[0] or x +
        half >= gray.shape[1]:
12         return None
13
14     mag, ang = gradient_maps(gray)
15     patch_mag = mag[y-half:y+half, x-half:x+half]
16     patch_ang = ang[y-half:y+half, x-half:x+half]
17

```

```

18 descriptor = []
19 bin_width = 360.0 / num_bins
20
21 for cy in range(0, patch_size, cell_size):
22     for cx in range(0, patch_size, cell_size):
23         cell_mag = patch_mag[cy:cy+cell_size, cx:cx+cell_size]
24         cell_ang = patch_ang[cy:cy+cell_size, cx:cx+cell_size]
25         hist = np.zeros(num_bins, dtype=np.float32)
26         for yy in range(cell_mag.shape[0]):
27             for xx in range(cell_mag.shape[1]):
28                 b = int(cell_ang[yy, xx] // bin_width) % num_bins
29                 hist[b] += cell_mag[yy, xx]
30             descriptor.extend(hist.tolist())
31
32 descriptor = np.array(descriptor, dtype=np.float32)
33 norm = np.linalg.norm(descriptor) + 1e-8
34 descriptor = descriptor / norm
35 descriptor = np.clip(descriptor, 0, 0.2)
36 descriptor = descriptor / (np.linalg.norm(descriptor) + 1e-8)
37 return descriptor
38
39 gray = cv2.imread("data/harris_building.jpg", cv2.IMREAD_GRAYSCALE).
40     astype(np.float32) / 255.0
41 if len(corners) > 0:
42     x, y, _ = corners[0]
43     desc = sift_like_descriptor(gray, x, y)
44     print("Keypoint location:", (x, y))
45     print("Descriptor shape:", desc.shape if desc is not None else None)
46     print("Descriptor first 32 values:", desc[:32] if desc is not None
47         else None)
48     print("Descriptor L2 norm:", np.linalg.norm(desc) if desc is not None
49         else None)

```

6.7. OpenCV SIFT implementation

Listing 6: Notebook Cell: OpenCV SIFT detectAndCompute

```

1 # Cell 6 - OpenCV SIFT
2 img_bgr = cv2.imread("data/sift_book.jpg")
3 gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
4
5 sift = cv2.SIFT_create()
6 keypoints, descriptors = sift.detectAndCompute(gray, None)
7
8 print("Number of SIFT keypoints:", len(keypoints))
9 print("Descriptor array shape:", descriptors.shape if descriptors is not
10     None else None)
11
12 for i, kp in enumerate(keypoints[:5]):
13     print(f"Keypoint {i}:")
14     print("  pt      =", kp.pt)
15     print("  size     =", kp.size)
16     print("  angle     =", kp.angle)
17     print("  response  =", kp.response)
18     print("  octave   =", kp.octave)
19     if descriptors is not None:
20         print("  desc[:16]=", descriptors[i][:16])
21
22 vis = cv2.drawKeypoints(

```

```

22     img_bgr, keypoints, None,
23     flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
24 )
25 show_image("SIFT keypoints", vis, figsize=(8, 6))

```

Expected results. On textured objects such as books, posters, logos, and architectural facades, SIFT typically yields many more practically useful matching features than raw corners plus patch descriptors. Students should observe that descriptor vectors are high-dimensional but structured: they are not arbitrary pixel values, but histograms encoding local gradient distributions.

Discussion. This is the point where the lab transitions from feature detection to feature representation. Students should understand that a strong detector without a robust descriptor is usually insufficient for correspondence. Conversely, even a strong descriptor can fail if the detector does not find repeatable locations.

Exercises after SIFT.

1. Compare patch descriptors with SIFT-like descriptors on a rotated image pair.
2. Print and inspect descriptor vectors from multiple nearby keypoints. Which dimensions change most?
3. Rotate an image by a known angle and measure how many SIFT matches remain valid.
4. Compare SIFT keypoints to DoG detections from your simplified implementation.

7. Feature matching

7.1. Distance between descriptors

Given two descriptors $\mathbf{d}_1$ and $\mathbf{d}_2$, a common similarity measure is the Euclidean distance:

$$\|\mathbf{d}_1 - \mathbf{d}_2\|_2.$$

A smaller distance indicates greater similarity.

7.2. Nearest-neighbor and kNN matching

The simplest strategy is nearest-neighbor matching: for each descriptor in image 1, find the closest descriptor in image 2. However, nearest-neighbor matching alone often yields ambiguous correspondences, especially in repetitive scenes.

A more reliable strategy is to look at the two nearest neighbors and apply *Lowe's ratio test*.

7.3. Lowe's ratio test

Let d_1 and d_2 be the distances to the first and second nearest neighbors. A match is kept only if

$$\frac{d_1}{d_2} < \tau,$$

where τ is often around 0.7 to 0.8.

The intuition is that a good match should be much better than the second-best alternative.

Listing 7: Notebook Cell: SIFT matching and Lowe ratio test

```

1 # Cell 7 - Matching with Lowe ratio test
2 img1 = cv2.imread("data/match_left.jpg")
3 img2 = cv2.imread("data/match_right.jpg")
4 gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
5 gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
6
7 sift = cv2.SIFT_create()
8 kp1, des1 = sift.detectAndCompute(gray1, None)
9 kp2, des2 = sift.detectAndCompute(gray2, None)
10
11 bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=False)
12 knn_matches = bf.knnMatch(des1, des2, k=2)
13
14 good_matches = []
15 for m, n in knn_matches:
16     if m.distance < 0.75 * n.distance:
17         good_matches.append(m)
18
19 print("Number of keypoints in image 1:", len(kp1))
20 print("Number of keypoints in image 2:", len(kp2))
21 print("Number of raw knn match pairs:", len(knn_matches))
22 print("Number of good matches after ratio test:", len(good_matches))
23
24 vis_matches = cv2.drawMatches(
25     img1, kp1, img2, kp2, good_matches[:80], None,
26     flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
27 )
28 show_image("Good SIFT matches", vis_matches, figsize=(14, 6))

```

7.4. What information does a match contain?

In OpenCV, a `DMatch` object stores:

- `queryIdx`: index of the descriptor in image 1,
- `trainIdx`: index of the descriptor in image 2,
- `distance`: descriptor dissimilarity.

These are not yet guaranteed to be geometrically correct correspondences.

Exercises after matching.

1. Compare raw nearest-neighbor matches and ratio-test-filtered matches.
2. Try ratio thresholds of 0.6, 0.7, 0.8, and 0.9. How does the number of matches change?
3. Visualize the top-20, top-50, and top-100 matches by distance.
4. Identify repeated structures where many wrong matches occur.

8. RANSAC for robust geometric estimation

8.1. Why RANSAC is needed

Even after descriptor filtering, many correspondences remain incorrect. If one estimates a geometric transformation using all matches blindly, a relatively small number of outliers can severely distort

the result. RANSAC, short for Random Sample Consensus, solves this by repeatedly hypothesizing a model from a minimal subset and testing how many correspondences agree with it.

8.2. Key ideas

RANSAC separates data into:

- **inliers**: points consistent with a model,
- **outliers**: points inconsistent with that model.

Instead of trying to fit all data at once, RANSAC seeks the model supported by the largest consensus set.

8.3. Homography model

For image alignment between two views of a planar scene or approximately rotational camera motion, the mapping between images can often be approximated by a homography:

$$\mathbf{x}' \sim H\mathbf{x},$$

where $\mathbf{x} = [x, y, 1]^T$, $\mathbf{x}' = [x', y', 1]^T$, and H is a 3×3 matrix defined up to scale.

A homography requires at least four point correspondences.

8.4. RANSAC pipeline

Algorithm 2 RANSAC for homography estimation

- 1: Input candidate correspondences
 - 2: **for** iteration = 1 to N **do**
 - 3: Randomly sample 4 correspondences
 - 4: Estimate candidate homography H
 - 5: Reproject all source points using H
 - 6: Compute reprojection error for each correspondence
 - 7: Mark points with error below threshold ϵ as inliers
 - 8: Keep the model with the largest inlier set
 - 9: **end for**
 - 10: Re-estimate homography using all inliers of the best model
 - 11: Return best homography and inlier mask
-

8.5. Reprojection error

If $\hat{\mathbf{x}}'_i$ is the point predicted by homography H and $\mathbf{x}'_i$ is the matched destination point, then a simple reprojection error is

$$e_i = \|\mathbf{x}'_i - \hat{\mathbf{x}}'_i\|_2.$$

A match is considered an inlier if $e_i < \epsilon$ for some threshold ϵ .

8.6. Synthetic NumPy demonstration

Listing 8: Notebook Cell: simple affine-style RANSAC demonstration on synthetic data

```

1 # Cell 8 - Simple RANSAC demonstration with synthetic 2D correspondences
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 rng = np.random.default_rng(0)
```

```

6
7 # Inlier correspondences from a hidden affine transform
8 N_in, N_out = 40, 20
9 pts1_in = rng.uniform(0, 100, size=(N_in, 2))
10
11 A_true = np.array([[1.1, 0.2, 12.0],
12                    [-0.1, 0.9, 18.0]], dtype=np.float32)
13
14 pts1_h = np.hstack([pts1_in, np.ones((N_in, 1), dtype=np.float32)])
15 pts2_in = (A_true @ pts1_h.T).T
16 pts2_in += rng.normal(scale=1.0, size=pts2_in.shape)
17
18 pts1_out = rng.uniform(0, 100, size=(N_out, 2))
19 pts2_out = rng.uniform(0, 130, size=(N_out, 2))
20
21 pts1 = np.vstack([pts1_in, pts1_out])
22 pts2 = np.vstack([pts2_in, pts2_out])
23
24 def fit_affine_least_squares(src, dst):
25     n = src.shape[0]
26     X = np.zeros((2*n, 6), dtype=np.float32)
27     y = np.zeros((2*n,), dtype=np.float32)
28     for i, ((x, y1), (u, v)) in enumerate(zip(src, dst)):
29         X[2*i] = [x, y1, 1, 0, 0, 0]
30         X[2*i+1] = [0, 0, 0, x, y1, 1]
31         y[2*i] = u
32         y[2*i+1] = v
33     params, *_ = np.linalg.lstsq(X, y, rcond=None)
34     return params.reshape(2, 3)
35
36 def affine_error(A, src, dst):
37     src_h = np.hstack([src, np.ones((len(src), 1), dtype=np.float32)])
38     pred = (A @ src_h.T).T
39     return np.linalg.norm(pred - dst, axis=1)
40
41 best_A = None
42 best_inliers = None
43 best_count = -1
44
45 for _ in range(500):
46     idx = rng.choice(len(pts1), size=3, replace=False)
47     A = fit_affine_least_squares(pts1[idx], pts2[idx])
48     err = affine_error(A, pts1, pts2)
49     inliers = err < 3.0
50     count = int(inliers.sum())
51     if count > best_count:
52         best_count = count
53         best_A = A
54         best_inliers = inliers
55
56 best_A_refined = fit_affine_least_squares(pts1[best_inliers], pts2[
57     best_inliers])
58
59 print("Best inlier count:", best_count)
60 print("Estimated affine matrix:\n", best_A_refined)
61
62 plt.figure(figsize=(6, 6))
63 plt.scatter(pts1[:,0], pts1[:,1], c=["green" if t else "red" for t in
64     best_inliers], label="source pts")
65 plt.title("Source points colored by RANSAC inlier mask")

```

```

64 plt.gca().invert_yaxis()
65 plt.legend()
66 plt.show()

```

8.7. OpenCV homography with RANSAC

Listing 9: Notebook Cell: homography estimation with OpenCV RANSAC

```

1 # Cell 9 - Homography with OpenCV RANSAC
2 if len(good_matches) >= 4:
3     src_pts = np.float32([kp1[m.queryIdx].pt for m in good_matches]).
4         reshape(-1, 1, 2)
5     dst_pts = np.float32([kp2[m.trainIdx].pt for m in good_matches]).
6         reshape(-1, 1, 2)
7
8     H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 4.0)
9     mask = mask.ravel().astype(bool)
10
11     inlier_matches = [m for i, m in enumerate(good_matches) if mask[i]]
12
13     print("Homography matrix:\n", H)
14     print("Matches before RANSAC:", len(good_matches))
15     print("Inliers after RANSAC:", int(mask.sum()))
16     print("Inlier ratio:", float(mask.mean()))
17
18     vis_inliers = cv2.drawMatches(
19         img1, kp1, img2, kp2, inlier_matches[:80], None,
20         flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
21     )
22     show_image("RANSAC inlier matches", vis_inliers, figsize=(14, 6))
23 else:
24     print("Not enough good matches to estimate homography.")

```

Expected results. Students should observe a clear reduction in geometrically inconsistent correspondences after RANSAC. The visual pattern of remaining lines should look more coherent, often concentrated on the overlapping physical structure shared between the two images.

Discussion. RANSAC does not improve descriptors, and it does not invent good matches from bad data. Its role is geometric verification. Therefore, if the initial matching quality is extremely poor, RANSAC may fail or produce unstable homographies.

Exercises after RANSAC.

1. Change the reprojection threshold and measure how the number of inliers changes.
2. Compare affine-based and homography-based geometric verification for the same image pair.
3. Create a failure example with too little overlap or too many repeated patterns.
4. Report the mean reprojection error on inliers and compare across image pairs.

9. Application: image alignment and basic panorama

9.1. Full pipeline overview

We can now connect all previous stages:

1. detect keypoints and descriptors in both images,
2. match descriptors,
3. reject ambiguous matches,
4. estimate homography with RANSAC,
5. warp one image into the coordinate frame of the other,
6. build a larger canvas and combine the images.

9.2. OpenCV panoramic alignment cell

Listing 10: Notebook Cell: basic panorama stitching with SIFT and RANSAC

```

1 # Cell 10 - Basic panorama stitching
2 img1 = cv2.imread("data/pano_left.jpg")
3 img2 = cv2.imread("data/pano_right.jpg")
4
5 gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
6 gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
7
8 sift = cv2.SIFT_create()
9 kp1, des1 = sift.detectAndCompute(gray1, None)
10 kp2, des2 = sift.detectAndCompute(gray2, None)
11
12 bf = cv2.BFMatcher(cv2.NORM_L2)
13 knn_matches = bf.knnMatch(des1, des2, k=2)
14 good = [m for m, n in knn_matches if m.distance < 0.75 * n.distance]
15
16 if len(good) < 4:
17     raise RuntimeError("Not enough good matches for panorama stitching.")
18
19 src_pts = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1,
20     2)
21 dst_pts = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1,
22     2)
23
24 H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 4.0)
25
26 h1, w1 = img1.shape[:2]
27 h2, w2 = img2.shape[:2]
28
29 corners_img1 = np.float32([[0,0],[w1,0],[w1,h1],[0,h1]]).reshape(-1,1,2)
30 warped_corners_img1 = cv2.perspectiveTransform(corners_img1, H)
31 corners_img2 = np.float32([[0,0],[w2,0],[w2,h2],[0,h2]]).reshape(-1,1,2)
32
33 all_corners = np.vstack((warped_corners_img1, corners_img2))
34 [xmin, ymin] = np.int32(all_corners.min(axis=0).ravel() - 0.5)
35 [xmax, ymax] = np.int32(all_corners.max(axis=0).ravel() + 0.5)
36
37 translation = np.array([[1, 0, -xmin],
38     [0, 1, -ymin],
39     [0, 0, 1]], dtype=np.float64)

```

```

38
39 result = cv2.warpPerspective(img1, translation @ H, (xmax - xmin, ymax -
40     ymin))
41 result[-ymin:h2-ymin, -xmin:w2-xmin] = img2
42 print("Good matches:", len(good))
43 print("RANSAC inliers:", int(mask.sum()))
44 show_image("Basic panorama result", result, figsize=(14, 7))

```

9.3. Failure cases to discuss

Students should explicitly analyze cases where the panorama is poor:

- too little overlap,
- many repeated structures,
- severe non-planarity,
- moving objects,
- large exposure differences,
- wide-baseline perspective distortion.

Expected results. On well-chosen image pairs with moderate overlap and sufficient texture, a basic panorama should visibly align shared regions. Seams and brightness mismatches may remain, since the current pipeline does not perform advanced blending.

Discussion. This final application is intentionally simple. Its purpose is to show how a complete classical vision system can emerge from fundamental components already introduced in the course: filtering, gradients, local structures, matching, and geometry.

10. Suggested structure of the accompanying notebook

The notebook should be organized into clearly separated cells. A recommended structure is:

Cell group	Purpose
0	Imports, helper functions, visualization utilities, side-by-side display
1	NumPy Harris implementation
2	OpenCV Harris implementation
3	Gaussian pyramid and DoG pyramid
4	Simple patch descriptor
5	Simplified SIFT-like descriptor
6	OpenCV SIFT detectAndCompute
7	Descriptor matching and Lowe ratio test
8	Synthetic RANSAC demonstration
9	Homography estimation with OpenCV RANSAC
10	Panorama stitching
11	Evaluation summary and analysis
12	Final exercises and submission checklist

11. Evaluation and reporting guidelines

Students should report both qualitative and quantitative observations:

- number of detected corners or keypoints,
- number of raw matches,
- number of ratio-test filtered matches,
- number of RANSAC inliers,
- inlier ratio,
- qualitative comments on robustness to scale, rotation, blur, and illumination.

A compact report table may contain columns such as:

Image pair	Keypoints	Good matches	Inliers	Inlier ratio
Pair 1	—	—	—	—
Pair 2	—	—	—	—
Pair 3	—	—	—	—

12. Comprehensive final exercises

Final exercises for the whole lab.

1. **Detector comparison study.** Compare Harris, DoG, and SIFT detections on the same image set. Analyze repeatability, spatial distribution, and practical usefulness for matching.
2. **Descriptor baseline study.** Use Harris points with simple normalized patches, then compare against SIFT descriptors on rotated and scaled image pairs.
3. **Robust alignment experiment.** For at least three image pairs, evaluate raw matching, ratio-test-filtered matching, and RANSAC-filtered matching. Report the number of correspondences at each stage.
4. **Panorama construction.** Build a basic panorama from two overlapping views. Discuss when the method succeeds and when it fails.
5. **Failure case analysis.** Intentionally choose difficult image pairs and explain why the pipeline breaks down. Relate the failure to detector quality, descriptor ambiguity, or geometric model mismatch.

13. Submission checklist

Students should submit:

- the completed notebook with all cells executed,
- a short report or exported PDF with selected visualizations,
- a set of test images used for experiments,
- answers to all exercise questions,
- a final discussion of at least one success case and one failure case.

14. Concluding remarks

This lab is designed to illustrate a central principle of classical computer vision: meaningful image understanding often begins with stable local structure. Harris demonstrates how local gradients reveal corners; LoG and DoG show why scale matters; SIFT turns local neighborhoods into robust descriptors; matching converts descriptors into tentative correspondences; RANSAC transforms noisy correspondences into a geometrically consistent model; and image alignment turns the whole pipeline into a visible application.

Although modern deep learning systems often learn features automatically, the concepts in this lab remain foundational. They sharpen geometric intuition, clarify what descriptors actually encode, and provide a robust baseline for many practical vision tasks.